

PRINTABLE PRO TRACK

GenAI Pro Extension Track

A clean printable guide for the production-scale layer: serving, deployment, security, caching, data flywheels, and distributed systems for GenAI.

150h

first-pass learning plus notes

6

pro modules from inference to distributed systems

3

senior-level build checkpoints

Use this after the core canon: this is the operate-it-at-scale layer that turns app-level GenAI skill into senior systems engineering evidence.

Quick Navigation

How This Track Relates To The Canon

Prerequisites

What This Track Adds (The Missing 25-30%)

Time Model

Pro Build Cadence

Pro Module P1: LLM Inference And Serving At Scale

Topic P1.1: Inference fundamentals and GPU economics

Topic P1.2: Serving engines and optimization techniques

Topic P1.3: Quantization, parallelism, and capacity planning

Pro Module P2: LLM Ops And Deployment Lifecycle

Topic P2.1: Versioning, registries, and reproducibility

Topic P2.2: Safe deployment strategies

Topic P2.3: CI/CD and operational maturity

Pro Module P3: Security And Responsible AI (Deep)

Topic P3.1: The GenAI threat model

Topic P3.2: Controls, isolation, and red-teaming

Topic P3.3: Governance, privacy, and compliance

Pro Module P4: Caching And Model Gateway Architecture

Topic P4.1: Caching strategies for GenAI

Topic P4.2: Model gateway and routing layer

Topic P4.3: Cost and reliability engineering

Pro Module P5: Data Flywheel And Continuous Improvement

Topic P5.1: Capturing the right signals

Topic P5.2: From signals to growing eval sets

Topic P5.3: Closing the loop with optimization

Pro Module P6: Distributed Systems For GenAI

Topic P6.1: Concurrency, queues, and backpressure

Topic P6.2: Scaling, state, and storage

Topic P6.3: Reliability and observability at scale

Pro Completion Definition

Final Outcome

This document is the **Pro Extension Track** for the [GenAI Mastery Canon](#).

It exists because the core canon (Modules 1-22) takes you to a strong **GenAI application engineer**, but stops short of the **operate-it-at-scale, keep-it-safe-cheap-and-reliable** layer that separates a senior application engineer from a **MAANG-level / staff GenAI systems engineer**.

This track does not replace anything in the canon. It is an **append-only pro layer** you run *after* (or interleaved with) the core modules once you can already build working GenAI systems.

How This Track Relates To The Canon

- The canon teaches you to **design and build** GenAI systems (retrieval, prompting, agents, evaluation, safety basics).
- This track teaches you to **serve, deploy, secure, cache, and continuously improve** those systems at production scale.
- Treat it as **Phase VI** that sits on top of the canon's Phase I-V.

Prerequisites

Before starting this track, you should have completed (or be comfortable with):

- Canon Module 1 (mental models), Module 6 (RAG foundations), Module 8 (evaluation), Module 9 (safety basics), Module 10 (agent fundamentals).
- At least one shipped project from the canon's build cadence (Projects 1-4).
- Basic comfort with containers, a cloud provider, and CI/CD. Unlike the core canon, **this track intentionally lifts the "no general infra" exclusion** — you cannot operate GenAI at scale while pretending distributed systems do not exist.

What This Track Adds (The Missing 25-30%)

Pro Module	Closes this gap	Slots in near canon module
P1 - LLM Inference & Serving At Scale	Serving infra, GPU economics, throughput	1.1.c, 2.2 (inference), 20
P2 - LLMOps & Deployment Lifecycle	Release engineering for prompts/models	8, 11.3
P3 - Security & Responsible AI (Deep)	Full threat model + governance + compliance	9, 13.3
P4 - Caching & Model Gateway Architecture	Cost/latency infra, multi-provider resilience	20, 1.1.c
P5 - Data Flywheel & Continuous Improvement	Compounding data advantage, distill loop	8, 18
P6 - Distributed Systems For GenAI	The infra substrate the canon excluded	cross-cutting

Time Model

- Pro Extension Track total: about **150 hours** of first-pass learning plus notes.
- Add 25% to 40% if you build alongside (strongly recommended for this track).
- At 10-12 hours per week, roughly **3 to 4 months** on top of the canon.

Pro Build Cadence

These builds turn the track into demonstrable senior-level evidence.

- After **P1** and **P4**: Build a **self-hosted inference service** with a model gateway, semantic cache, and a measured cost/latency report vs a hosted baseline.
 - After **P2** and **P5**: Build a **prompt/model CI/CD pipeline** with offline eval gates, canary rollout, automated rollback, and a data-capture-to-eval-set loop.
 - After **P3** and **P6**: Build a **hardened, multi-tenant GenAI service** with defense-in-depth against the OWASP LLM Top 10, audit logging, and a documented scaling/failure design.
-

Pro Module P1: LLM Inference And Serving At Scale

Module time: 32h

Why this module matters: You cannot claim "scalable production GenAI" if you only know how to call a hosted API. This module is the difference between renting inference and owning it economically. It is also the single biggest gap in most self-taught GenAI engineers.

Topic P1.1: Inference fundamentals and GPU economics

Topic time: 10h

- The two phases of LLM inference: prefill vs decode, and why they cost differently - 2.5h
- GPU memory math: weights, KV cache, activations, and batch size limits - 2.5h
- Throughput vs latency vs cost: the inference iron triangle - 2.5h
- Hardware mental models: VRAM, memory bandwidth, and what actually bottlenecks - 2.5h

Topic P1.2: Serving engines and optimization techniques

Topic time: 12h

- vLLM, TGI, and TensorRT-LLM: what they optimize and when to choose each - 3h
- Continuous (in-flight) batching and paged attention intuition - 3h
- KV cache management, prefix caching, and reuse - 3h
- Speculative decoding, chunked prefill, and other latency tricks - 3h

Topic P1.3: Quantization, parallelism, and capacity planning

Topic time: 10h

- Quantization mental models: FP16/BF16, INT8, FP8, GPTQ, AWQ, and quality tradeoffs - 2.5h
- Tensor, pipeline, and data parallelism for large models - 2.5h
- Autoscaling, cold starts, and warm-pool strategies for GPU services - 2.5h
- Capacity planning: tokens/sec targets, concurrency, and headroom - 2.5h

Module checkpoint:

- Estimate the GPU memory and rough throughput for serving a given open-weight model.
 - Justify vLLM vs TGI vs a hosted API for a specific workload using cost and latency.
 - Explain how continuous batching and KV cache reuse change real throughput.
-

Pro Module P2: LLMOps And Deployment Lifecycle

Module time: 26h

Why this module matters: In real teams, the dangerous part is not writing a prompt, it is changing one in production without breaking users. This module is the release-engineering discipline that the canon's evaluation module hints at but does not fully operationalize.

Topic P2.1: Versioning, registries, and reproducibility

Topic time: 8h

- Versioning prompts, models, datasets, and eval sets together - 2h
- Model and prompt registries: promotion stages and metadata - 2h
- Reproducible runs: pinning model versions, seeds, and configs - 2h
- Environment parity: dev, staging, and prod for GenAI systems - 2h

Topic P2.2: Safe deployment strategies

Topic time: 10h

- Offline eval gates as a merge requirement - 2.5h
- Canary, shadow (mirror), and blue-green deployments for LLM changes - 2.5h
- Online A/B testing and guardrail metrics with statistical significance - 2.5h
- Automated rollback triggers on quality, latency, cost, and safety regressions - 2.5h

Topic P2.3: CI/CD and operational maturity

Topic time: 8h

- Building a prompt/model CI pipeline (lint, eval, regression, gate) - 2h
- Feature flags and dynamic config for fast, reversible changes - 2h
- Incident response runbooks for GenAI services - 2h
- Change management and approval flows for high-risk model updates - 2h

Module checkpoint:

- Design a deployment pipeline where a prompt change cannot reach prod without passing eval gates.
 - Explain canary vs shadow vs blue-green for an LLM system and when each fits.
 - Define concrete automated rollback triggers tied to measurable thresholds.
-

Pro Module P3: Security And Responsible AI (Deep)

Module time: 28h

Why this module matters: The canon's Module 9 introduces safety; this module takes it to the depth an enterprise or MAANG security review actually demands. At scale, security and governance are not features, they are gating requirements.

Topic P3.1: The GenAI threat model

Topic time: 10h

- OWASP LLM Top 10 walkthrough with concrete examples - 2.5h
- Direct vs indirect (retrieval/tool) prompt injection and defense-in-depth - 2.5h
- Data poisoning, training-data extraction, and supply-chain risks - 2.5h
- Output-handling vulnerabilities: insecure rendering, SSRF, code execution - 2.5h

Topic P3.2: Controls, isolation, and red-teaming

Topic time: 10h

- Layered defenses: input filters, allowlists, sandboxing, and human gates - 2.5h
- Tenant isolation, least-privilege tools, and permission-aware retrieval - 2.5h
- Secret management and action-confirmation for high-impact tools - 2.5h
- Red-teaming and adversarial evaluation as a continuous practice - 2.5h

Topic P3.3: Governance, privacy, and compliance

Topic time: 8h

- PII detection, redaction, and data-residency handling - 2h
- Audit logging, traceability, and explainability for regulators - 2h
- Model cards, data sheets, and responsible-AI documentation - 2h
- Bias, fairness, and harm evaluation with realistic limits - 2h

Module checkpoint:

- Walk through the OWASP LLM Top 10 and map each to a concrete defense in your system.
 - Explain why indirect prompt injection cannot be fixed at the prompt layer alone.
 - Describe the governance artifacts an enterprise review would require before launch.
-

Pro Module P4: Caching And Model Gateway Architecture

Module time: 22h

Why this module matters: Caching and a gateway layer are frequently 30-60% cost reduction and a major reliability upgrade in real systems, yet most learners never build them. This is high-leverage, interview-relevant infrastructure.

Topic P4.1: Caching strategies for GenAI

Topic time: 8h

- Exact-match response caching and cache key design - 2h
- Semantic caching with embeddings and similarity thresholds - 2h
- Provider prompt caching and prefix reuse - 2h
- Cache invalidation, staleness, and correctness risks - 2h

Topic P4.2: Model gateway and routing layer

Topic time: 8h

- Why a gateway (LiteLLM-style) exists: one interface, many providers - 2h
- Model routing, fallback tiers, and dynamic quality/cost tiers - 2h
- Rate-limit handling, quotas, retries, and request hedging - 2h
- Multi-provider and multi-region failover for resilience - 2h

Topic P4.3: Cost and reliability engineering

Topic time: 6h

- FinOps for GenAI: cost per request, per session, per successful task - 2h
- Budget enforcement, throttling, and graceful degradation - 2h
- Observability for caches and gateways: hit rate, savings, fallbacks - 2h

Module checkpoint:

- Design a semantic cache and explain its correctness vs savings tradeoff.
 - Justify a model gateway and describe its routing and failover behavior.
 - Quantify the cost and reliability impact of caching plus gateway on a workload.
-

Pro Module P5: Data Flywheel And Continuous Improvement

Module time: 22h

Why this module matters: The compounding advantage of a production GenAI system is its **data loop**, not its model. This module turns usage into an ever-improving asset and connects evaluation (Module 8) to optimization (Module 18).

Topic P5.1: Capturing the right signals

Topic time: 8h

- Logging prompts, contexts, outputs, traces, and outcomes safely - 2h
- Implicit vs explicit feedback and their reliability - 2h
- Privacy-safe capture: consent, redaction, and retention - 2h
- Turning production failures into reproducible fixtures - 2h

Topic P5.2: From signals to growing eval sets

Topic time: 8h

- Triaging and labeling captured data into golden sets - 2h
- Hard-negative mining and edge-case curation - 2h
- Detecting drift and growing coverage where the system is weak - 2h
- Keeping eval sets trustworthy as they scale - 2h

Topic P5.3: Closing the loop with optimization

Topic time: 6h

- When the loop justifies distillation or fine-tuning vs prompt/retrieval fixes - 2h
- Synthetic data generation and curation pitfalls - 2h
- Measuring whether the loop actually improved the system - 2h

Module checkpoint:

- Design a data-capture pipeline that respects privacy and produces reusable eval cases.
 - Explain how production failures become regression fixtures and grow coverage.
 - Decide when the flywheel justifies fine-tuning vs cheaper retrieval/prompt fixes.
-

Pro Module P6: Distributed Systems For GenAI

Module time: 20h

Why this module matters: The core canon deliberately excluded general infra to stay focused. At the pro tier that exclusion must be lifted, because every scaling, latency, and reliability decision in GenAI is ultimately a distributed-systems decision.

Topic P6.1: Concurrency, queues, and backpressure

Topic time: 7h

- Async request handling and streaming at scale - 2.5h
- Queues, worker pools, and backpressure for spiky LLM traffic - 2.5h
- Timeouts, retries, idempotency, and the thundering-herd problem - 2h

Topic P6.2: Scaling, state, and storage

Topic time: 7h

- Horizontal scaling, load balancing, and stateless service design - 2.5h
- Where state lives: sessions, memory stores, and vector DB scaling - 2.5h
- Consistency, partitioning, and multitenancy at the data layer - 2h

Topic P6.3: Reliability and observability at scale

Topic time: 6h

- SLOs, error budgets, and circuit breakers for GenAI services - 2h
- Distributed tracing (OpenTelemetry-style) across the GenAI stack - 2h
- Capacity, failure injection, and chaos basics for LLM systems - 2h

Module checkpoint:

- Explain how backpressure and queues protect a GenAI service under spikes.
 - Describe what stays stateless and where state must live, and why.
 - Define SLOs and circuit-breaker behavior for a production GenAI endpoint.
-

Pro Completion Definition

You can claim the **MAANG-level / staff GenAI systems** identity when, in addition to the canon's completion definition, you can do all of the following without bluffing:

- Size, deploy, and economically justify a self-hosted inference service vs a hosted API.
- Ship a prompt or model change through eval gates, canary, and automated rollback.
- Defend a GenAI system against the OWASP LLM Top 10 with layered, enforceable controls.
- Cut cost and improve resilience with caching and a model gateway, and prove the impact.
- Run a privacy-safe data flywheel that turns production usage into measurable improvement.
- Reason about every scaling and reliability decision as a distributed-systems decision.

Final Outcome

If you complete the canon **and** this Pro Extension Track properly, your market identity becomes:

A GenAI systems engineer who can not only design and build retrieval, orchestration, evaluation, and agent systems, but also serve them at scale, deploy them safely, secure them against real threats, run them affordably, and improve them continuously, with the distributed-systems judgment to defend every decision in a senior design review.